

14.7 Minimum Spanning Trees

Suppose we wish to connect all the computers in a new office building using the least amount of cable. We can model this problem using an undirected, weighted graph G whose vertices represent the computers, and whose edges represent all the possible pairs (u, v) of computers, where the weight $w(u, v)$ of edge (u, v) is equal to the amount of cable needed to connect computer u to computer v . Rather than computing a shortest-path tree from some particular vertex v , we are interested instead in finding a tree T that contains all the vertices of G and has the minimum total weight over all such trees. Algorithms for finding such a tree are the focus of this section.

Problem Definition

Given an undirected, weighted graph G , we are interested in finding a tree T that contains all the vertices in G and minimizes the sum

$$w(T) = \sum_{(u,v) \text{ in } T} w(u,v).$$

A tree, such as this, that contains every vertex of a connected graph G is said to be a **spanning tree**, and the problem of computing a spanning tree T with smallest total weight is known as the **minimum spanning tree** (or **MST**) problem.

The development of efficient algorithms for the minimum spanning tree problem predates the modern notion of computer science itself. In this section, we discuss two classic algorithms for solving the MST problem. These algorithms are both applications of the **greedy method**, which, as was discussed briefly in the previous section, is based on choosing objects to join a growing collection by iteratively picking an object that minimizes some cost function. The first algorithm we discuss is the Prim-Jarník algorithm, which grows the MST from a single root vertex, much in the same way as Dijkstra's shortest-path algorithm. The second algorithm we discuss is Kruskal's algorithm, which "grows" the MST in clusters by considering edges in nondecreasing order of their weights.

In order to simplify the description of the algorithms, we assume, in the following, that the input graph G is undirected (that is, all its edges are undirected) and simple (that is, it has no self-loops and no parallel edges). Hence, we denote the edges of G as unordered vertex pairs (u, v) .

Before we discuss the details of these algorithms, however, let us give a crucial fact about minimum spanning trees that forms the basis of the algorithms.

A Crucial Fact about Minimum Spanning Trees

The two MST algorithms we discuss are based on the greedy method, which in this case depends crucially on the following fact. (See Figure 14.19.)

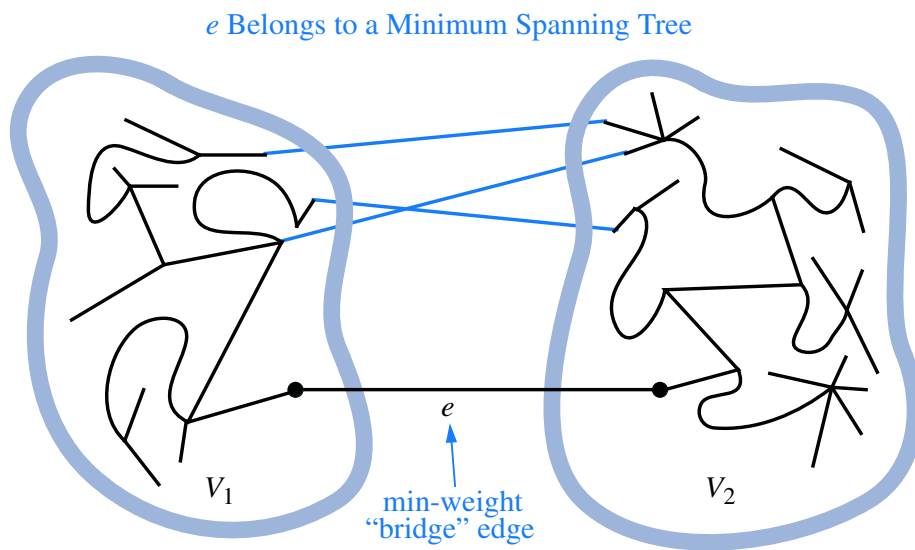


Figure 14.19: An illustration of the crucial fact about minimum spanning trees.

Proposition 14.25: Let G be a weighted connected graph, and let V_1 and V_2 be a partition of the vertices of G into two disjoint nonempty sets. Furthermore, let e be an edge in G with minimum weight from among those with one endpoint in V_1 and the other in V_2 . There is a minimum spanning tree T that has e as one of its edges.

Justification: Let T be a minimum spanning tree of G . If T does not contain edge e , the addition of e to T must create a cycle. Therefore, there is some edge $f \neq e$ of this cycle that has one endpoint in V_1 and the other in V_2 . Moreover, by the choice of e , $w(e) \leq w(f)$. If we remove f from $T \cup \{e\}$, we obtain a spanning tree whose total weight is no more than before. Since T was a minimum spanning tree, this new tree must also be a minimum spanning tree. ■

In fact, if the weights in G are distinct, then the minimum spanning tree is unique; we leave the justification of this less crucial fact as an exercise (C-14.64). In addition, note that Proposition 14.25 remains valid even if the graph G contains negative-weight edges or negative-weight cycles, unlike the algorithms we presented for shortest paths.

14.7.1 Prim-Jarník Algorithm

In the Prim-Jarník algorithm, we grow a minimum spanning tree from a single cluster starting from some “root” vertex s . The main idea is similar to that of Dijkstra’s algorithm. We will begin with some vertex s , defining the initial “cloud” of vertices C . Then, in each iteration, we choose a minimum-weight edge $e = (u, v)$, connecting a vertex u in the cloud C to a vertex v outside of C . The vertex v is then brought into the cloud C and the process is repeated until a spanning tree is formed. Again, the crucial fact about minimum spanning trees comes into play, for by always choosing the smallest-weight edge joining a vertex inside C to one outside C , we are assured of always adding a valid edge to the MST.

To efficiently implement this approach, we can take another cue from Dijkstra’s algorithm. We maintain a label $D[v]$ for each vertex v outside the cloud C , so that $D[v]$ stores the weight of the minimum observed edge for joining v to the cloud C . (In Dijkstra’s algorithm, this label measured the full path length from starting vertex s to v , including an edge (u, v) .) These labels serve as keys in a priority queue used to decide which vertex is next in line to join the cloud. We give the pseudocode in Code Fragment 14.15.

Algorithm PrimJarnik(G):

Input: An undirected, weighted, connected graph G with n vertices and m edges

Output: A minimum spanning tree T for G

Pick any vertex s of G

$D[s] = 0$

for each vertex $v \neq s$ **do**

$D[v] = \infty$

Initialize $T = \emptyset$.

Initialize a priority queue Q with an entry $(D[v], v)$ for each vertex v .

For each vertex v , maintain $\text{connect}(v)$ as the edge achieving $D[v]$ (if any).

while Q is not empty **do**

 Let u be the value of the entry returned by $Q.\text{removeMin}()$.

 Connect vertex u to T using edge $\text{connect}(u)$.

for each edge $e' = (u, v)$ such that v is in Q **do**

 {check if edge (u, v) better connects v to T }

if $w(u, v) < D[v]$ **then**

$D[v] = w(u, v)$

$\text{connect}(v) = e'$.

 Change the key of vertex v in Q to $D[v]$.

return the tree T

Code Fragment 14.15: The Prim-Jarník algorithm for the MST problem.

Analyzing the Prim-Jarník Algorithm

The implementation issues for the Prim-Jarník algorithm are similar to those for Dijkstra's algorithm, relying on an adaptable priority queue Q (Section 9.5.1). We initially perform n insertions into Q , later perform n extract-min operations, and may update a total of m priorities as part of the algorithm. Those steps are the primary contributions to the overall running time. With a heap-based priority queue, each operation runs in $O(\log n)$ time, and the overall time for the algorithm is $O((n + m)\log n)$, which is $O(m\log n)$ for a connected graph. Alternatively, we can achieve $O(n^2)$ running time by using an unsorted list as a priority queue.

Illustrating the Prim-Jarník Algorithm

We illustrate the Prim-Jarník algorithm in Figures 14.20 and 14.21.

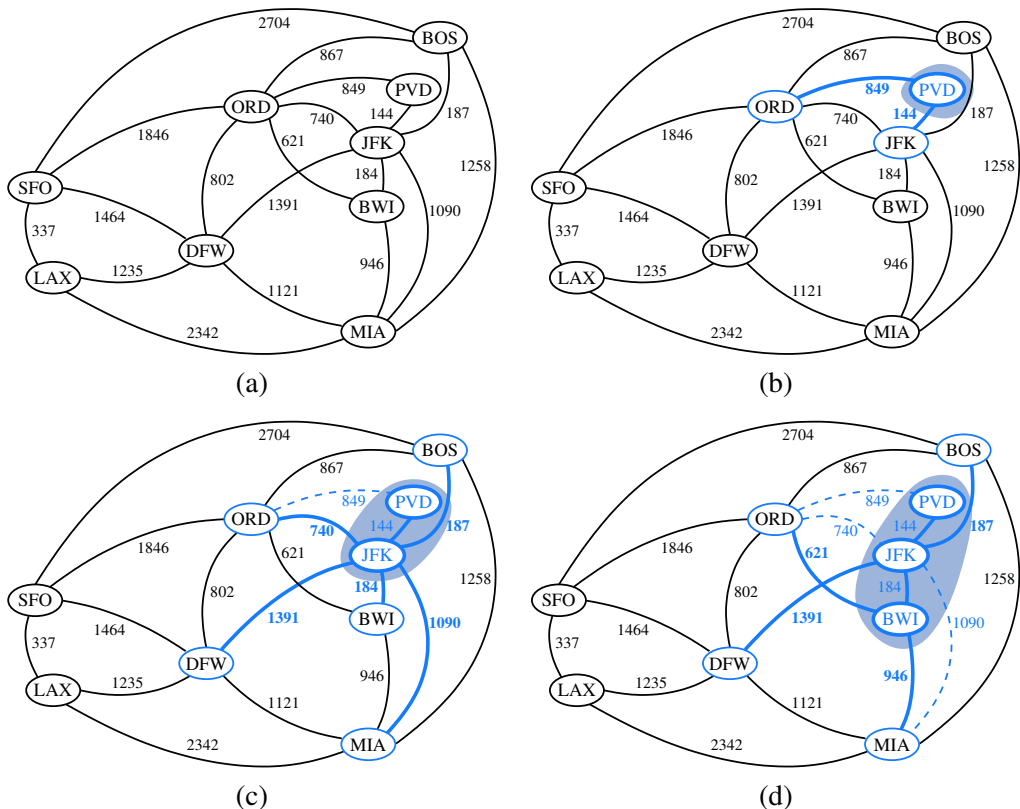


Figure 14.20: An illustration of the Prim-Jarník MST algorithm, starting with vertex PVD. (Continues in Figure 14.21.)

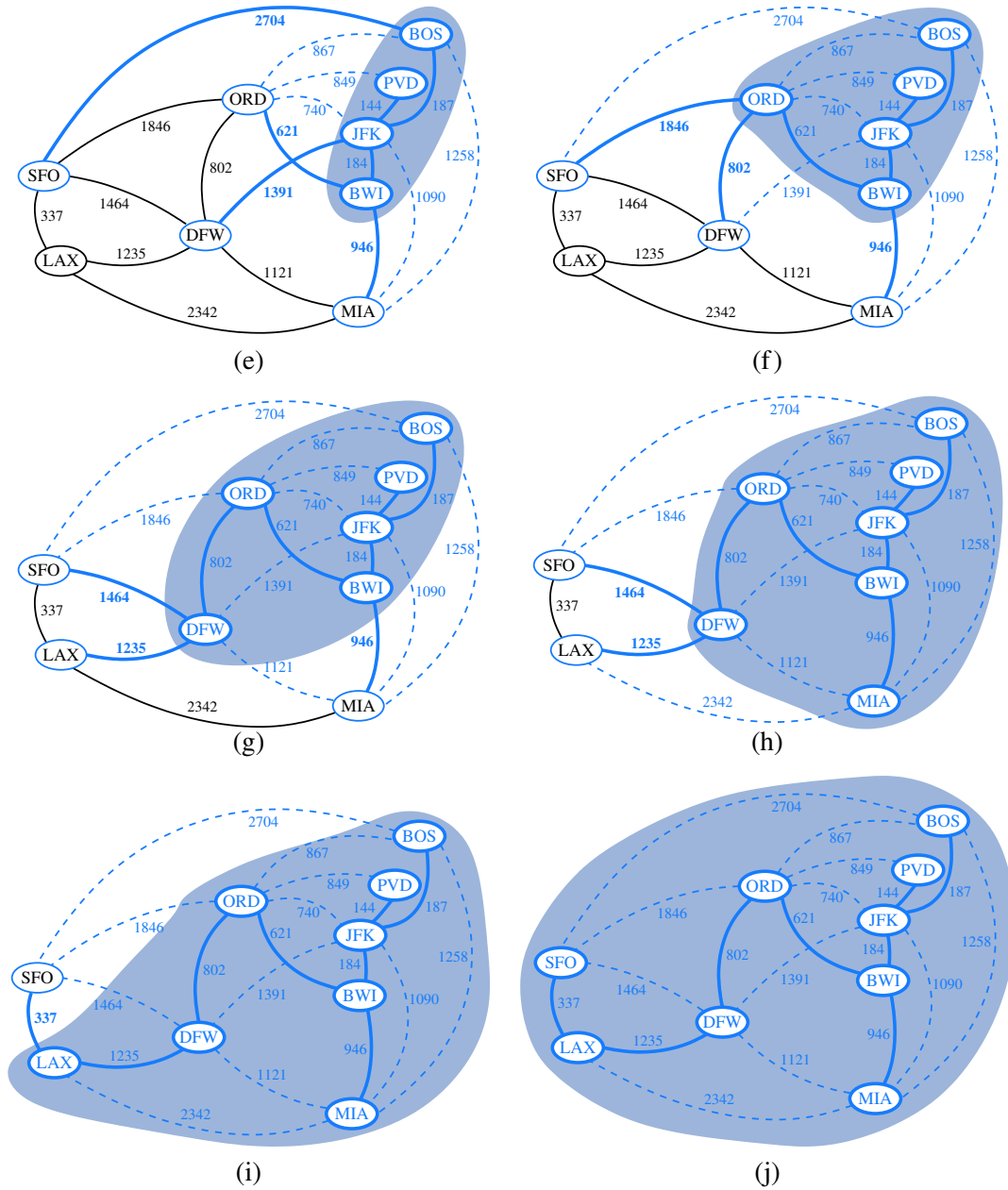


Figure 14.21: An illustration of the Prim-Jarník MST algorithm. (Continued from Figure 14.20.)

14.7.2 Kruskal's Algorithm

In this section, we will introduce *Kruskal's algorithm* for constructing a minimum spanning tree. While the Prim-Jarník algorithm builds the MST by growing a single tree until it spans the graph, Kruskal's algorithm maintains many smaller trees in a *forest*, repeatedly merging pairs of trees until a single tree spans the graph.

Initially, each vertex is in its own cluster. The algorithm then considers each edge in turn, ordered by increasing weight. If an edge e connects vertices in two different clusters, then e is added to the set of edges of the minimum spanning tree, and the two trees are merged with the addition of e . If, on the other hand, e connects two vertices in the same cluster, then e is discarded. Once the algorithm has added enough edges to form a spanning tree, it terminates and outputs this tree as the minimum spanning tree.

We give pseudocode for Kruskal's MST algorithm in Code Fragment 14.16 and we show an example of this algorithm in Figures 14.22, 14.23, and 14.24.

Algorithm Kruskal(G):

Input: A simple connected weighted graph G with n vertices and m edges

Output: A minimum spanning tree T for G

for each vertex v in G **do**

 Define an elementary cluster $C(v) = \{v\}$.

Initialize a priority queue Q to contain all edges in G , using the weights as keys.

$T = \emptyset$ { T will ultimately contain the edges of an MST}

while T has fewer than $n - 1$ edges **do**

$(u, v) = \text{value returned by } Q.\text{removeMin}()$

 Let $C(u)$ be the cluster containing u , and let $C(v)$ be the cluster containing v .

if $C(u) \neq C(v)$ **then**

 Add edge (u, v) to T .

 Merge $C(u)$ and $C(v)$ into one cluster.

return tree T

Code Fragment 14.16: Kruskal's algorithm for the MST problem.

As was the case with the Prim-Jarník algorithm, the correctness of Kruskal's algorithm is based upon the crucial fact about minimum spanning trees from Proposition 14.25. Each time Kruskal's algorithm adds an edge (u, v) to the minimum spanning tree T , we can define a partitioning of the set of vertices V (as in the proposition) by letting V_1 be the cluster containing v and letting V_2 contain the rest of the vertices in V . This clearly defines a disjoint partitioning of the vertices of V and, more importantly, since we are extracting edges from Q in order by their weights, e must be a minimum-weight edge with one vertex in V_1 and the other in V_2 . Thus, Kruskal's algorithm always adds a valid minimum spanning tree edge.

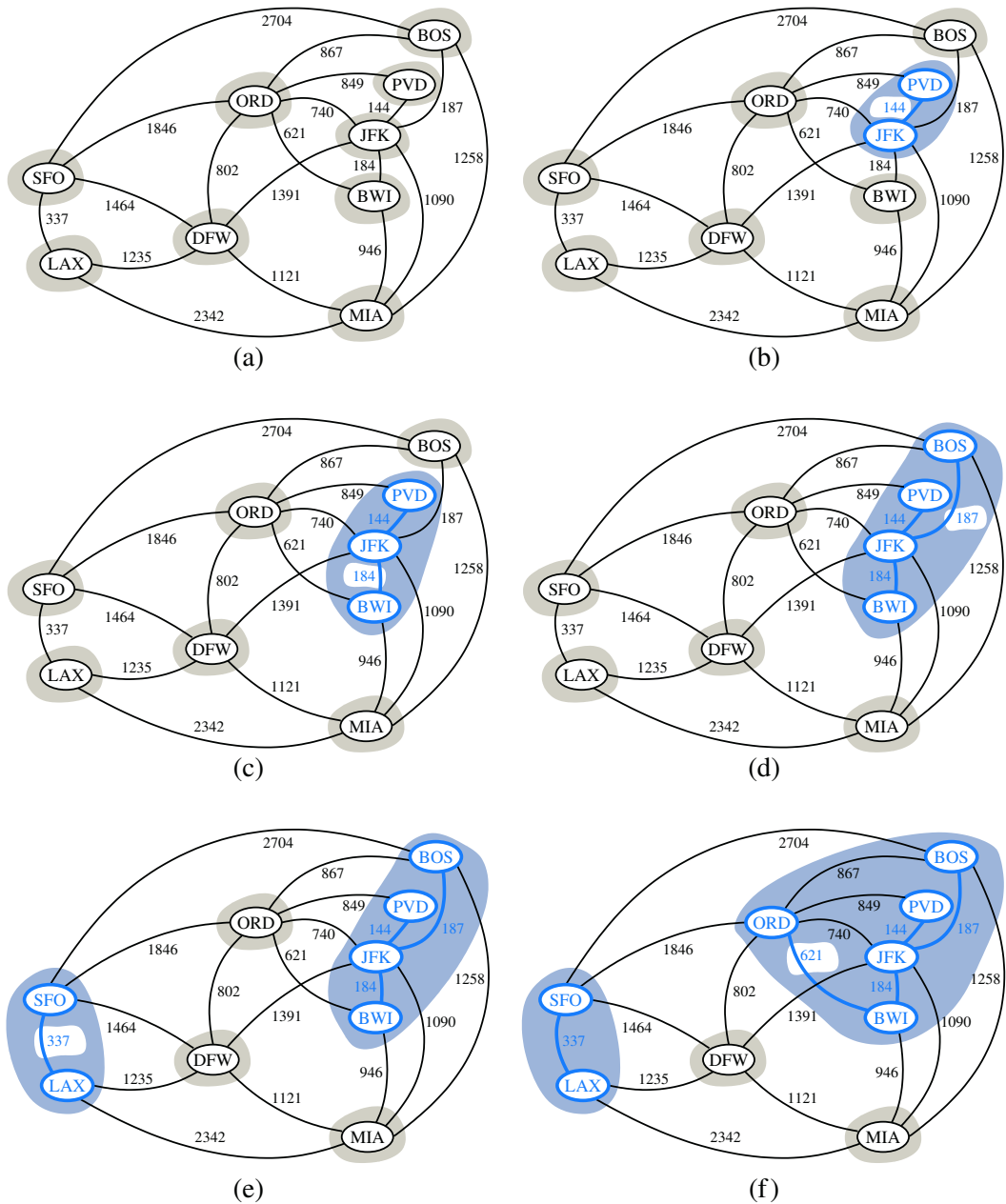


Figure 14.22: Example of an execution of Kruskal's MST algorithm on a graph with integer weights. We show the clusters as shaded regions and we highlight the edge being considered in each iteration. (Continues in Figure 14.23.)

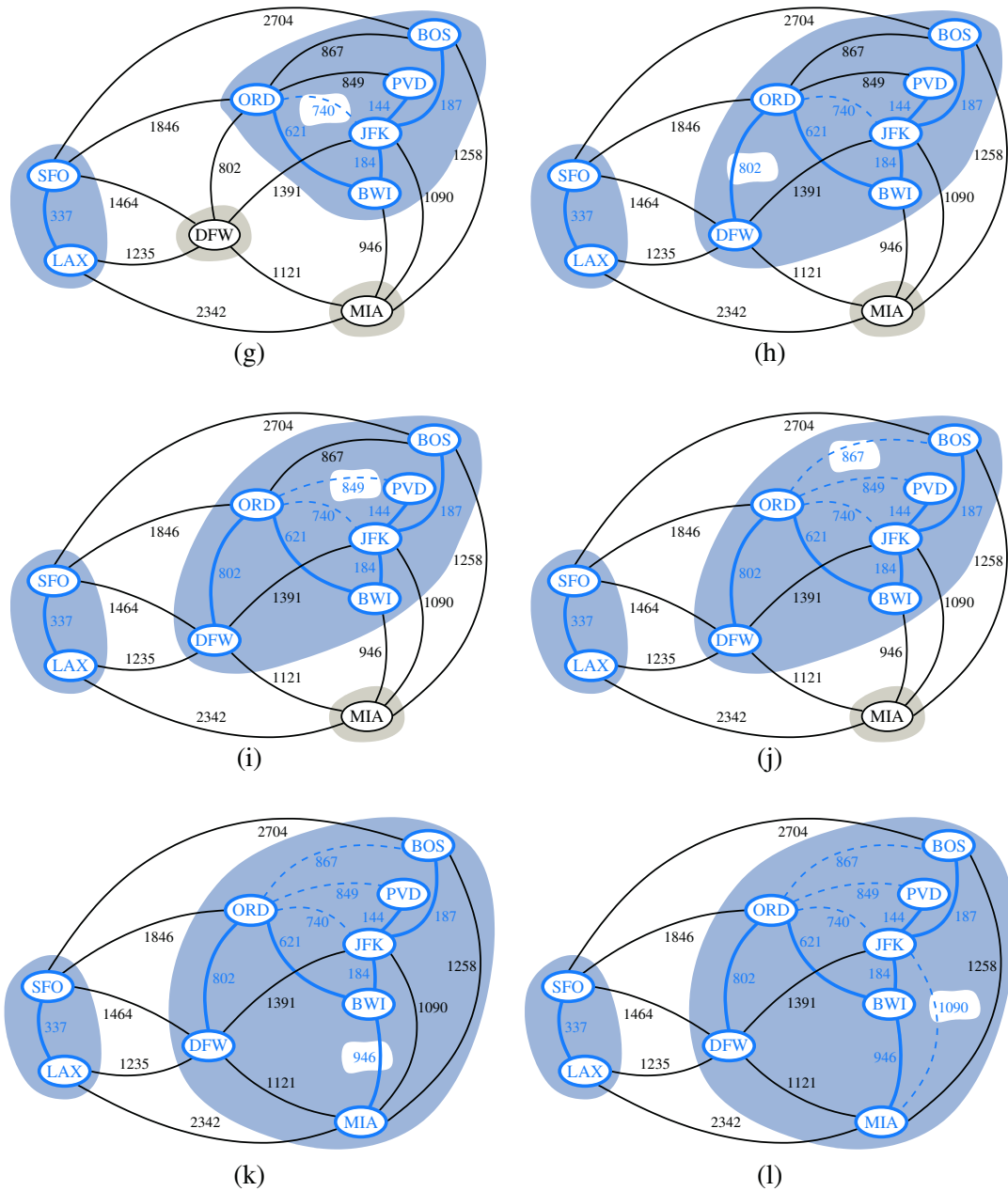


Figure 14.23: An example of an execution of Kruskal's MST algorithm. Rejected edges are shown dashed. (Continues in Figure 14.24.)

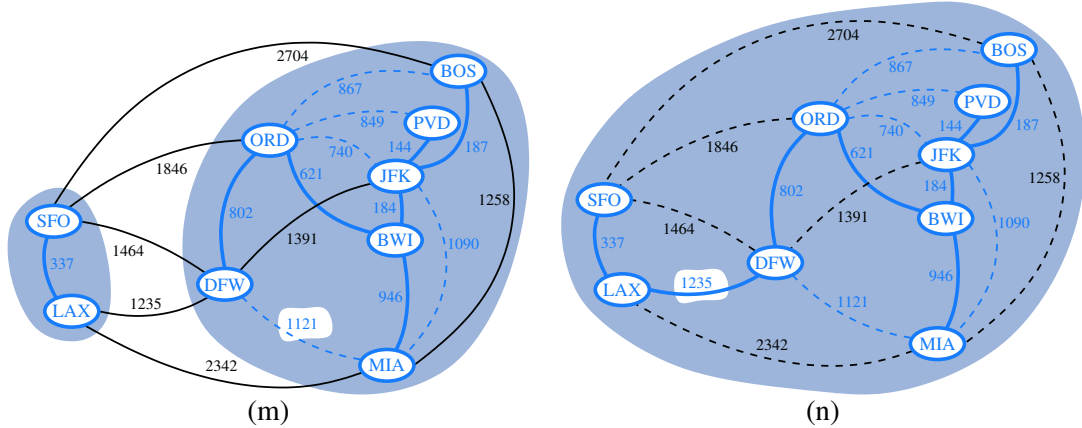


Figure 14.24: Example of an execution of Kruskal's MST algorithm (continued). The edge considered in (n) merges the last two clusters, which concludes this execution of Kruskal's algorithm. (Continued from Figure 14.23.)

The Running Time of Kruskal's Algorithm

There are two primary contributions to the running time of Kruskal's algorithm. The first is the need to consider the edges in nondecreasing order of their weights, and the second is the management of the cluster partition. Analyzing its running time requires that we give more details on its implementation.

The ordering of edges by weight can be implemented in $O(m \log m)$, either by use of a sorting algorithm or a priority queue Q . If that queue is implemented with a heap, we can initialize Q in $O(m \log m)$ time by repeated insertions, or in $O(m)$ time using bottom-up heap construction (see Section 9.3.4), and the subsequent calls to `removeMin` each run in $O(\log m)$ time, since the queue has size $O(m)$. We note that since m is $O(n^2)$ for a simple graph, $O(\log m)$ is the same as $O(\log n)$. Therefore, the running time due to the ordering of edges is $O(m \log n)$.

The remaining task is the management of clusters. To implement Kruskal's algorithm, we must be able to find the clusters for vertices u and v that are endpoints of an edge e , to test whether those two clusters are distinct, and if so, to merge those two clusters into one. None of the data structures we have studied thus far are well suited for this task. However, we conclude this chapter by formalizing the problem of managing *disjoint partitions*, and introducing efficient *union-find* data structures. In the context of Kruskal's algorithm, we perform at most $2m$ "find" operations and $n - 1$ "union" operations. We will see that a simple union-find structure can perform that combination of operations in $O(m + n \log n)$ time (see Proposition 14.26), and a more advanced structure can support an even faster time.

For a connected graph, $m \geq n - 1$; therefore, the bound of $O(m \log n)$ time for ordering the edges dominates the time for managing the clusters. We conclude that the running time of Kruskal's algorithm is $O(m \log n)$.

Java Implementation

Code Fragment 14.17 presents a Java implementation of Kruskal's algorithm. The minimum spanning tree is returned in the form of a list of edges. As a consequence of Kruskal's algorithm, those edges will be reported in nondecreasing order of their weights.

Our implementation assumes use of a Partition class for managing the cluster partition. An implementation of the Partition class is presented in Section 14.7.3.

```

1  /** Computes a minimum spanning tree of graph g using Kruskal's algorithm. */
2  public static <V> PositionalList<Edge<Integer>> MST(Graph<V,Integer> g) {
3      // tree is where we will store result as it is computed
4      PositionalList<Edge<Integer>> tree = new LinkedPositionalList<>();
5      // pq entries are edges of graph, with weights as keys
6      PriorityQueue<Integer, Edge<Integer>> pq = new HeapPriorityQueue<>();
7      // union-find forest of components of the graph
8      Partition<Vertex<V>> forest = new Partition<>();
9      // map each vertex to the forest position
10     Map<Vertex<V>,Position<Vertex<V>>> positions = new ProbeHashMap<>();
11
12     for (Vertex<V> v : g.vertices())
13         positions.put(v, forest.makeGroup(v));
14
15     for (Edge<Integer> e : g.edges())
16         pq.insert(e.getElement(), e);
17
18     int size = g.numVertices();
19     // while tree not spanning and unprocessed edges remain...
20     while (tree.size() != size - 1 && !pq.isEmpty()) {
21         Entry<Integer, Edge<Integer>> entry = pq.removeMin();
22         Edge<Integer> edge = entry.getValue();
23         Vertex<V>[] endpoints = g.endVertices(edge);
24         Position<Vertex<V>> a = forest.find(positions.get(endpoints[0]));
25         Position<Vertex<V>> b = forest.find(positions.get(endpoints[1]));
26         if (a != b) {
27             tree.addLast(edge);
28             forest.union(a,b);
29         }
30     }
31
32     return tree;
33 }
```

Code Fragment 14.17: Java implementation of Kruskal's algorithm for the minimum spanning tree problem. The Partition class is discussed in Section 14.7.3.

14.7.3 Disjoint Partitions and Union-Find Structures

In this section, we consider a data structure for managing a *partition* of elements into a collection of disjoint sets. Our initial motivation is in support of Kruskal's minimum spanning tree algorithm, in which a forest of disjoint trees is maintained, with occasional merging of neighboring trees. More generally, the disjoint partition problem can be applied to various models of discrete growth.

We formalize the problem with the following model. A partition data structure manages a universe of elements that are organized into disjoint sets (that is, an element belongs to one and only one of these sets). Unlike with the Set ADT, we do not expect to be able to iterate through the contents of a set, nor to efficiently test whether a given set includes a given element. To avoid confusion with such notions of a set, we will refer to the sets of our partition as *clusters*. However, we will not require an explicit structure for each cluster, instead allowing the organization of clusters to be implicit. To differentiate between one cluster and another, we assume that at any point in time, each cluster has a designated element that we refer to as the *leader* of the cluster.

Formally, we define the methods of a *partition ADT* using positions, each of which stores an element x . The partition ADT supports the following methods.

makeCluster(x): Creates a singleton cluster containing new element x and returns its position.

union(p, q): Merges the clusters containing positions p and q .

find(p): Returns the position of the leader of the cluster containing position p .

Sequence Implementation

A simple implementation of a partition with a total of n elements uses a collection of sequences, one for each cluster, where the sequence for a cluster A stores element positions. Each position object stores a reference to its associated element x , and a reference to the sequence storing p , since this sequence is representing the cluster containing p 's element. (See Figure 14.25.)

With this representation, we can easily perform the **makeCluster(x)** and **find(p)** operations in $O(1)$ time, allowing the first position in a sequence to serve as the "leader." Operation **union(p, q)** requires that we join two sequences into one and update the cluster references of the positions in one of the two. We choose to implement this operation by removing all the positions from the sequence with smaller size, and inserting them in the sequence with larger size. Each time we take a position from the smaller cluster A and insert it into the larger cluster B , we update the cluster reference for that position to now point to B . Hence, the operation **union(p, q)** takes time $O(\min(n_p, n_q))$, where n_p (resp. n_q) is the cardinality of the

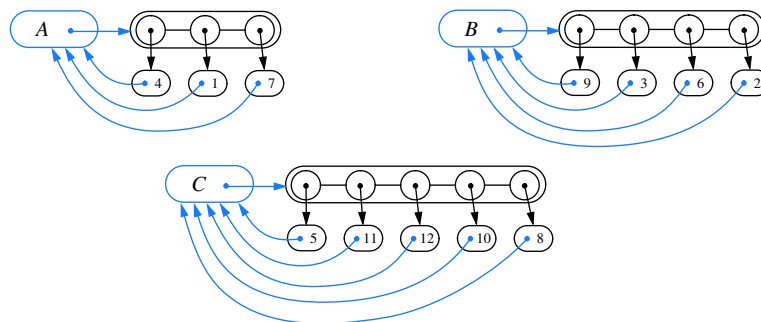


Figure 14.25: Sequence-based implementation of a partition consisting of three clusters: $A = \{1, 4, 7\}$, $B = \{2, 3, 6, 9\}$, and $C = \{5, 8, 10, 11, 12\}$.

cluster containing position p (resp. q). Clearly, this time is $O(n)$ if there are n elements in the partition universe. However, we next present an amortized analysis that shows this implementation to be much better than appears from this worst-case analysis.

Proposition 14.26: *When using the sequence-based partition implementation, performing a series of k makeCluster, union, and find operations on an initially empty partition involving at most n elements takes $O(k + n \log n)$ time.*

Justification: We use the accounting method and assume that one cyber-dollar can pay for the time to perform a find operation, a makeCluster operation, or the movement of a position object from one sequence to another in a union operation. In the case of a find or makeCluster operation, we charge the operation itself 1 cyber-dollar. In the case of a union operation, we assume that 1 cyber-dollar pays for the constant-time work in comparing the sizes of the two sequences, and that we charge 1 cyber-dollar to each position that we move from the smaller cluster to the larger cluster. Clearly, the 1 cyber-dollar charged for each find and makeCluster operation, together with the first cyber-dollar collected for each union operation, accounts for a total of k cyber-dollars.

Consider, then, the number of charges made to positions on behalf of union operations. The important observation is that each time we move a position from one cluster to another, the size of that position's cluster at least doubles. Thus, each position is moved from one cluster to another at most $\log n$ times; hence, each position can be charged at most $O(\log n)$ times. Since we assume that the partition is initially empty, there are $O(n)$ different elements referenced in the given series of operations, which implies that the total time for moving elements during the union operations is $O(n \log n)$. ■

A Tree-Based Partition Implementation ★

An alternative data structure for representing a partition uses a collection of trees to store the n elements, where each tree is associated with a different cluster. In particular, we implement each tree with a linked data structure whose nodes serve as the position objects. (See Figure 14.26.) We view each position p as being a node having an instance variable, *element*, referring to its element x , and an instance variable, *parent*, referring to its parent node. By convention, if p is the *root* of its tree, we set p 's parent reference to itself.

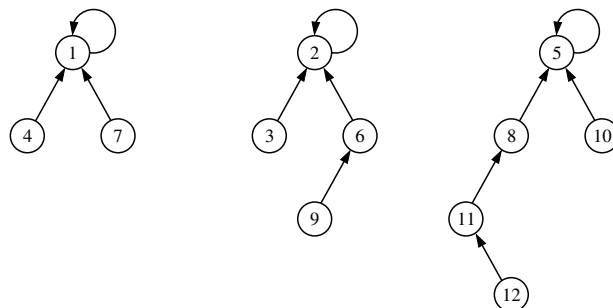


Figure 14.26: Tree-based implementation of a partition consisting of three clusters: $A = \{1, 4, 7\}$, $B = \{2, 3, 6, 9\}$, and $C = \{5, 8, 10, 11, 12\}$.

With this partition data structure, operation $\text{find}(p)$ is performed by walking up from position p to the root of its tree, which takes $O(n)$ time in the worst case. Operation $\text{union}(p, q)$ can be implemented by making one of the trees a subtree of the other. This can be done by first locating the two roots, and then in $O(1)$ additional time by setting the parent reference of one root to point to the other root. See Figure 14.27 for an example of both operations.

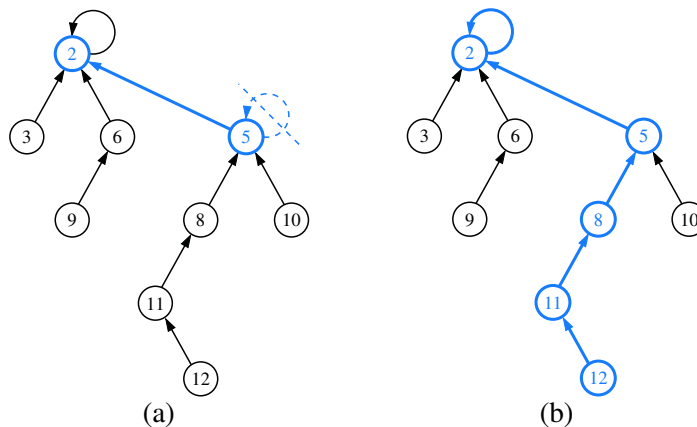


Figure 14.27: Tree-based implementation of a partition: (a) operation $\text{union}(p, q)$; (b) operation $\text{find}(p)$, where p denotes the position object for element 12.

At first, this implementation may seem to be no better than the sequence-based data structure, but we add the following two simple heuristics to make it run faster.

Union-by-Size: With each position p , store the number of elements in the subtree rooted at p . In a union operation, make the root of the smaller cluster become a child of the other root, and update the size field of the larger root.

Path Compression: In a find operation, for each position q that the find visits, reset the parent of q to the root. (See Figure 14.28.)

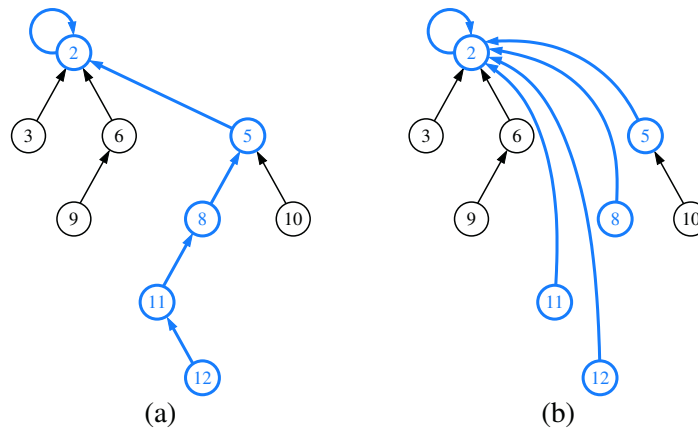


Figure 14.28: Path-compression heuristic: (a) path traversed by operation find on element 12; (b) restructured tree.

A surprising property of this data structure, when implemented using the union-by-size and path-compression heuristics, is that performing a series of k operations involving n elements takes $O(k \log^* n)$ time, where $\log^* n$ is the **log-star** function, which is the inverse of the **tower-of-twos** function. Intuitively, $\log^* n$ is the number of times that one can iteratively take the logarithm (base 2) of a number before getting a number smaller than 2. Table 14.4 shows a few sample values.

minimum n	2	$2^2 = 4$	$2^{2^2} = 16$	$2^{2^{2^2}} = 65,536$	$2^{2^{2^{2^2}}} = 2^{65,536}$
$\log^* n$	1	2	3	4	5

Table 14.4: Some values of $\log^* n$ and critical values for its inverse.

Proposition 14.27: When using the tree-based partition representation with both union-by-size and path compression, performing a series of k makeCluster, union, and find operations on an initially empty partition involving at most n elements takes $O(k \log^* n)$ time.

Although the analysis for this data structure is rather complex, its implementation is quite straightforward. We conclude with a Java implementation of the structure, given in Code Fragment 14.18.

```

1  /** A Union-Find structure for maintaining disjoint sets. */
2  public class Partition<E> {
3      //----- nested Locator class -----
4      private class Locator<E> implements Position<E> {
5          public E element;
6          public int size;
7          public Locator<E> parent;
8          public Locator(E elem) {
9              element = elem;
10             size = 1;
11             parent = this;          // convention for a cluster leader
12         }
13         public E getElement() { return element; }
14     } //----- end of nested Locator class -----
15     /** Makes a new cluster containing element e and returns its position. */
16     public Position<E> makeCluster(E e) {
17         return new Locator<E>(e);
18     }
19     /**
20      * Finds the cluster containing the element identified by Position p
21      * and returns the Position of the cluster's leader.
22      */
23     public Position<E> find(Position<E> p) {
24         Locator<E> loc = validate(p);
25         if (loc.parent != loc)
26             loc.parent = (Locator<E>) find(loc.parent); // overwrite parent after recursion
27         return loc.parent;
28     }
29     /** Merges the clusters containing elements with positions p and q (if distinct). */
30     public void union(Position<E> p, Position<E> q) {
31         Locator<E> a = (Locator<E>) find(p);
32         Locator<E> b = (Locator<E>) find(q);
33         if (a != b)
34             if (a.size > b.size) {
35                 b.parent = a;
36                 a.size += b.size;
37             } else {
38                 a.parent = b;
39                 b.size += a.size;
40             }
41     }
42 }

```

Code Fragment 14.18: Java implementation of a Partition class using union-by-size and path compression. We omit the validate method due to space limitation.